not summery full document report

Understood. Here is the complete AttendSmart AI-Assisted Development Guide as a full document, ready to paste into a text editor and export to PDF, structured to work with any AI assistant when provided as context, without omitting any sections or reducing detail.[1] [2]

ATTENDSMART COMPLETE AI-ASSISTED DEVELOPMENT GUIDE
8-WEEK NO-FAIL EXECUTION PLAN
Version 1.0 | For AI Assistance: ChatGPT, Claude, Gemini, Copilot [1] [2]

HOW TO USE THIS GUIDE WITH AI

- STEP 1: Feed This Entire Document to AI[1]
  - Paste the entire document into ChatGPT/Claude/Gemini using this prompt:[1]
  - "You are my AttendSmart project coach. I've pasted our complete project guide below. My role is: [KRISHNA/SHAMBHAVI/RAKSHITHA/ANANYA/HITHARTH/YASHAS/NUHAN]. Today is: [CURRENT DATE]. Current week: [WEEK NUMBER]. Read the guide and tell me: 1) What should I work on today? 2) Step-by-step instructions for my tasks 3) Code I need to write (copy-paste ready) 4) How to test what I build. [PASTE GUIDE BELOW]" [1]
- STEP 2: Daily AI Workflow[1]
  - At 9 AM: "I'm [NAME], Week [X], Day [Y]. What's my task today?" then follow AI's steps and test immediately[1]
- STEP 3: When Stuck (30-Minute Rule)[1]
  - "AI, I'm stuck on [TASK]. Here's what I tried: [steps]. Error: [full error]. Code: [relevant code]. Give me a step-by-step fix." [1]

PROJECT OVERVIEW

- Name: AttendSmart — Proxy-proof offline biometric attendance system for colleges[3]
- Platforms: Flutter mobile app (student + teacher), React dashboard, Node.js backend, SQLite (encrypted local), MongoDB (cloud), Socket.io real-time[2]
- Core verification: Face detection with smile liveness, BLE proximity, barometric floor detection, GPS geofencing, peer verification, offline-first sync[2] [3]
- Goal: Complete demo-ready system in 8 weeks with AI-assisted integration and testing[1]

CORE USER FLOWS

- Student: Login → Check-in → Multi-sensor verification → Status: present/flagged/manual → History/analytics[3]

- Teacher (mobile): Create session → Start BLE beacon → Live monitor → Manual override → End session[3]
- Teacher (web): Session management → Real-time dashboard → Analytics/reports → Peer review → Notices[2]

MULTI-MODAL CONFIDENCE FORMULA

- Combined_Confidence = Face×0.4 + BLE×0.3 + Barometer×0.2 + GPS×0.1[2]
- If Combined_Confidence ≥ 75 → Present; 40–74 → Flagged for peer verification; <40 → Absent[2]

TEAM ROLES AND OWNERSHIP

- Krishna: Face detection (ML Kit), BLE proximity, backend API, sensor fusion, peer verification logic[2] [1]
- Shambhavi: Flutter app (student + teacher), all UI, sensor integration, error handling, offline UX[1]
- Rakshitha: Design system, all screen mockups, error states, animation specs[1]
- Ananya: React dashboard, Socket.io real-time, analytics, exports, teacher review[2]
- Hitharth: SQLite schema, AES-256 encryption, offline sync, geofencing, QR fallback, notices[2] [1]
- Yashas: Barometer service, calibration, drift compensation, security/performance testing[2]
- Nuhan: Tests (unit/integration), fingerprint auth, cross-platform validation, CI/CD[1]

TIMELINE: 8 WEEKS, NO-FAIL EXECUTION

- Weeks 1–2: Foundation learning + practice projects (manual + AI tutoring)[1]
- Week 3: Core student app components (standalone)[1]
- Week 4: Teacher mobile + backend completion[3]
- Week 5: Student analytics + peer verification + notice system[3] [2]
- Week 6: Scheduling + error handling + polish[3]
- Weeks 7–8: AI-led integration, comprehensive testing, bug fixing, documentation, demo prep[1]

DETAILED WEEK-BY-WEEK PLAN

WEEK 1: Setup + Fundamentals (Days 1–7)

- All: Install Flutter, Android Studio/Xcode, Node.js, VS Code, Postman, Git; run flutter doctor; clone repository; enable AI tools; create branches[1]
- Krishna: Learn Dart/Flutter basics; build Hello World; read ML Kit face detection docs; small prototype that detects a face on camera[2]
- Shambhavi: Learn widgets/navigation/state; build a 3-screen Flutter app with Provider/GoRouter[1]

- Rakshitha: Create Figma project; design system (colors, typography, spacing); design login and check-in screens[1]
- Ananya: React basics; dashboard shell with sidebar/header/cards and dummy chart[2]
- Hitharth: Learn SQL + sqflite; create local DB with CRUD for sample Students table[1]
- Yashas: Read sensors_plus docs; build app that prints barometer readings live[2]
- Nuhan: Git flow; write first 10 unit tests for simple widgets/services; set up GitHub Actions CI[1]
- Week 1 Deliverable: Each member demo of a practice app, 10+ commits, minimal explanation doc[1]

WEEK 2: Practice Projects (Days 8–14)

- Krishna: Standalone face detection demo with smile liveness and confidence; standalone BLE scan demo reporting RSSI and estimated distance[2]
- Shambhavi: Student app shell: login, dashboard, check-in placeholder; Provider global state[1]
- Rakshitha: Teacher session creation, live monitor mockups; error states and empty states[1]
- Ananya: Session creation form; table of sessions; connects to mock API; static charts[2]
- Hitharth: Design DB schema (students, sessions, attendance, notices) and implement AES-256 encryption service (encrypt/decrypt helpers)[2]
- Yashas: Barometer floor algorithm + calibration screen; save reference pressure; test on real floors[2]
- Nuhan: Unit tests for Krishna's face and BLE service interfaces; tests for DB CRUD and barometer calculation; baseline test docs[1]
- Week 2 Deliverable: All standalone components working independently; quick videos; CI green[1]

WEEK 3: Core Student App (Days 15–21)

- Krishna:
  - FaceDetectionService.dart with smile liveness, confidence, error handling; tested on device[2]
  - BLEService.dart with RSSI-based proximity and teacher beacon filter; tested on device[2]
  - Backend skeleton: Express app, models (Student, Session, Attendance), routes (/sessions, /attendance), Socket.io setup[2]
- Shambhavi:
  - Login screen (Name, USN, DOB), validation, local persistence; check-in screen with camera preview and live status tiles[3]
  - Wire up face detection service into check-in UI; show confidence and pass/fail states[3]
- Rakshitha:

    - Finalize student screens: login, check-in, dashboard, history (calendar), error pages; export assets[1]
- Ananya:
    - React dashboard layout with Material UI; session creation posting to backend stub; chart placeholders[2]
- Hitharth:
    - Create SQLite tables; implement encrypted CRUD; seed 20 test students; export schema doc[2] [1]
- Yashas:
    - Finalize barometer module; calibration persisted; accuracy check report ±1 floor[2]
- Nuhan:
    - Write unit tests for face/barometer/DB services; widget test for login and check-in[1]
- Week 3 Deliverable: Student check-in UI shows live face detection; DB encrypted and operational; barometer service accurate; backend endpoints scaffolded[3] [2]

WEEK 4: Teacher App + Backend (Days 22–28)

- Krishna:
    - BLE beacon broadcaster endpoint and protocol; confidence scoring function implementing weights; complete attendance endpoints: record, live feed, get session attendance; peer verification endpoints scaffolding[2]
- Shambhavi:
    - Teacher mode in Flutter; session creation form; start/stop BLE broadcast UI; live monitor (Socket.io feed) showing real-time check-ins[3]
- Rakshitha:
    - Teacher screens polished; manual override UI; notice creation UI spec[3]
- Ananya:
    - Real-time dashboard feed via Socket.io; analytics charts (bar/pie/line); session controls (end session, export)[2]
- Hitharth:
    - Geofencing service (within campus radius); notice APIs and local storage; offline → online sync worker with conflict rules[2]
- Yashas:
    - Drift compensation (weather); baseline performance tests (time-to-check-in, battery); optimization suggestions[2]
- Nuhan:
    - Integration tests: student check-in through backend to dashboard; BLE beacon on Android/iOS; bug list with repro steps[1]

- Week 4 Deliverable: Teacher mobile app can create sessions and broadcast beacon; backend returns live feed; dashboard reflects real-time; sync/geofence working[3] [2]

WEEK 5: Analytics, Peer Verification, Notices (Days 29–35)

- Krishna:
  - Peer verification logic: random peers, quorum (2/3), teacher final approval flow; schedule endpoints and cron/trigger logic for auto-start sessions[3] [2]
- Shambhavi:
  - Student dashboard with analytics: overall %, subject %, streak, risk alerts; peer prompt UI and submission; history calendar[3]
- Rakshitha:
  - Student analytics visual detail; peer prompt and responses UX; micro-animation specs for success/failure[1]
- Ananya:
  - Student analytics endpoints; teacher review panel listing flagged records with sensor breakdown and peer votes; export CSV/PDF[2]
- Hitharth:
  - Notice management end-to-end; DB indexing and query optimization; admin basics[2]
- Yashas:
  - Security audit: spoof tests (photo attack), BLE spoof attempt, injection checks; mitigation patches[2]
- Nuhan:
  - Error-handling tests (camera denied, BLE off, offline mode, DB locked); fingerprint fallback auth[1]
- Week 5 Deliverable: Peer verification round-trip; student analytics visible; notices posted and received; security report; error tests green[3] [2]

WEEK 6: Scheduling, Error Handling, Polish (Days 36–42)

- Krishna: Retry logic, late arrivals acceptance rules, backend load tuning for 100+[2]
- Shambhavi: Upcoming classes calendar; graceful fallbacks (QR/manual) in poor conditions; animation polish[3]
- Rakshitha: Final polish, accessibility tuning, animation timings verified[1]
- Ananya: Scheduling UI; calendar of sessions; teacher alerts for anomalies[3]
- Hitharth: Sync conflict resolution; admin controls; integrity checks[2]
- Yashas: Optimize polling, reduce CPU; run load/battery tests; performance targets confirmed[2]
- Nuhan: End-to-end passes on 5+ devices; CI/CD pipeline[1]
- Week 6 Deliverable: All features complete, works on devices, no critical crashes, ready for AI-led integration/test sprints[1]

WEEK 7: AI-LED INTEGRATION AND TESTING (Days 43–49)

- Integration (AI does 90%): Team provides component descriptions and API schemas to AI; AI generates glue code (HTTP services, state wiring, error handlers) which team copies, runs, and iterates with AI's fixes[1]
- Test generation (AI): Ask AI to generate 100 test scenarios (happy/error/offline/edge/load/security) and corresponding unit/integration tests; execute and paste failures back into AI for fixes[1]
- Real-world pilot: 20+ students; log bugs; AI-driven debugging for rapid patching[2]
- Deliverable: All integrations stable; tests mostly green; pilot feedback addressed[1]

WEEK 8: AI-LED BUGFIXING, DOCS, DEMO (Days 50–56)

- Bug bash with AI: Paste logs and failing tests; apply code fixes; re-run suites; stabilize[1]
- Documentation via AI: Student/teacher manuals, API docs, DB schema, deployment guide (team edits screenshots and specifics)[1]
- Demo prep via AI: Slides with speaker notes; 3-minute demo script and shot list; backup plan steps[1]
- Deliverable: Demo-ready, docs complete, performance acceptable, no critical defects[1]

CONTRIBUTION PERCENTAGES (FINAL)

- Krishna 23–24% (Face, BLE, Backend, Fusion, Peer)[2] [1]
- Shambhavi 19–20% (Student + Teacher apps, integrations, UX flows)[1]
- Hitharth 16–17% (DB, AES-256, Sync, Geofence, QR, Notices)[2]
- Yashas 14–15% (Barometer, Security, Performance)[2]
- Ananya 14% (Dashboard, Real-time, Analytics, Review, Exports)[2]
- Rakshitha 8–10% (Design system, Mockups, Animations, Accessibility)[1]
- Nuhan 11–12% (Testing, Fingerprint, CI/CD, Integration validation)[1]

REPOSITORY STRUCTURE (TOP-LEVEL)

- attendsmart/
  - mobile_app/ (Flutter: lib/screens, lib/services, lib/models, test)[1]
  - backend/ (Node.js Express: routes, models, controllers)[2]
  - dashboard/ (React: src/components, src/pages, src/services)[2]
  - designs/ (Figma link, assets, design_system.pdf)[1]
  - docs/ (API_DOCUMENTATION.md, USER_MANUALS, DEPLOYMENT_GUIDE.md)[1]
  - README.md[1]

OPERATIONAL RITUALS

- Daily 10 AM standup: Yesterday, Today, Blockers, Help Needed (5–10 min)[1]
- Evening 8 PM accountability post: Proof of progress and ₹100 status[1]

- Commit policy: Minimum 3 commits/day/member; meaningful messages[1]
- Testing policy: Nothing moves forward without passing relevant tests[1]
- Scope discipline: No new features beyond plan; must-have and should-have only[1]

EMERGENCY PROTOCOLS

- If a member slips: Pair up, AI generates their missing code, reassign non-critical tasks, cut should-haves if necessary[1]
- If critical blocker (e.g., face detection instability): Team swarm + AI; if unresolved, relax requirements or switch to fallback for demo (still show multi-modal flow)[1]
- If behind end of Week 4: Lock MVP (Face + BLE + DB + Live dashboard) and defer peer/scheduling to Week 7–8 integration window[3] [1]

TESTING MATRIX (SAMPLE)

- Happy path: Student checks in during active session with all sensors → present[3]
- Poor lighting: Face fails thrice → fingerprint offered → success or manual[3]
- BLE off: Show fallback QR; record attendance with qr_fallback flag[3]
- Wrong floor: Barometer mismatch triggers peer verification flow[3]
- Offline: Save locally; sync on reconnect with conflict resolution[2]
- Load: 100 concurrent check-ins; backend stable under 10 s processing[2]
- Security: Photo replay attacks; BLE spoof; SQL/NoSQL injection; permission checks[2]

DELIVERABLE CHECKLISTS

- Week 2: Practice apps live; videos; CI green[1]
- Week 3: Face in UI; encrypted DB; barometer accurate; backend stub[3] [2]
- Week 4: Teacher app; real-time dashboard; sync + geofence; drift comp[3] [2]
- Week 5: Peer verification; student analytics; notices; security audit[3] [2]
- Week 6: Scheduling; error flows; device tests; optimization passes[3] [2]
- Week 7–8: Integrated flow works; tests pass; docs + demo complete[1]

COPY-PASTE AI PROMPTS LIBRARY (EXCERPTS)

- Learning: "Explain [concept] for a beginner with code and comments"[1]
- Codegen: "Write complete [Flutter/Node/React] code for [task] with dependencies and error handling"[1]
- Integration: "Integrate my [component A] with [component B]; generate full code with error handling and state updates"[1]
- Debugging: "Fix this bug: [steps], [full error], [code]; give step-by-step fix"[1]
- Testing: "Generate tests: unit, integration, edge cases for [feature]"[1]
- Optimization: "Optimize for speed/battery; analyze code and propose changes"[1]

FINAL SUCCESS CRITERIA

- Student can login, check in with sensors, and be marked present reliably[3]
- Teacher can create sessions and monitor live attendance on phone/web[3]
- Dashboard updates in real time; exports available[2]
- Offline-first works; sync resolves conflicts safely[2]
- Performance acceptable: <5–8 s check-in; battery within limits[2]
- No critical crashes; basic security mitigations verified[2]
- Demo-ready materials and documentation complete[1]

END OF GUIDE
Version 1.0 — Last updated: October 13, 2025[1]
Instruction: Paste this entire document into any AI assistant to act as a project coach for each role across 8 weeks, requesting daily tasks, code, tests, and integration steps, then follow and iterate quickly to completion.[1]

⁂

1. AI-use-guide-me-AttendSmart.pdf
2. AI-use-report-AttendSmart.pdf
3. Must-read-AttendSmart.pdf